

往年简答题（附答案）一览

2018年

问题1

请根据所学算法设计内容，简要描述你对“问题同构”的理解，并举例说明。

答案

两个看似不同的问题，在抽象层面上具有相同的数学模型和计算结构，可以使用相同的算法思想或技术来解决。

举例：**矩阵链乘** 与 **多边形最优三角剖分**

- 矩阵链乘：给定 n 个矩阵，求乘积的最少乘法次数
- 多边形三角剖分：给定 $n + 1$ 个顶点的凸多边形，求权值最小的三角剖分
- 两者都具有相同的递归结构和状态转移方程

问题2

请解释什么是最优子结构性质，并举例哪些算法设计思想要求问题具有最优子结构性质。

答案

最优子结构性质是指一个问题的最优解包含其子问题的最优解。换句话说，可以通过组合子问题的最优解来构造原问题的最优解。

动态规划算法和贪心算法要求问题具有最优子结构性质。

举例：

动态规划：0-1背包问题的最优子结构性质

设 $dp[i][c]$ 表示考虑前 i 个物品，背包容量为 c 时的最大价值。则最优子结构体现在：

1. 如果不选第 i 个物品，则 $dp[i][c] = dp[i - 1][c]$ ，即前 $i - 1$ 个物品在容量 c 下的最优解。
2. 如果选第 i 个物品（需满足 $w_i \leq c$ ），则 $dp[i][c] = dp[i - 1][c - w_i] + v_i$ ，即前 $i - 1$ 个物品在剩余容量 $c - w_i$ 下的最优解加上 v_i 。

因此，原问题的最优解 $dp[n][W]$ 由子问题 $dp[i - 1][\cdot]$ 的最优解组合而来，具有最优子结构。

贪心算法：贪心背包问题的最优子结构性质

设当前背包剩余容量为 c ，已获得价值为 V 。选择单位价值最高的物品 i ，装入 $\min(w_i, c)$ 的重量。则剩余子问题为：容量 $c' = c - \min(w_i, c)$ ，物品集合去掉已完全装入的物品（若物品有剩余则仍保留部分）。原问题的最优解 $V +$ 剩余子问题的最优解。

由于物品可分割，该贪心选择能保证每一步的局部最优解导致全局最优解，满足最优子结构。

问题3

请根据自己的理解，举例说明拉斯维加斯算法和蒙特卡罗算法的特点。

答案

拉斯维加斯算法：

特点：

1. **总是正确**：如果算法返回一个解，那么这个解一定是正确的
2. **运行时间随机**：运行时间是不确定的，但期望时间有限
3. **可能不终止**：理论上可能永远运行，但概率极低
4. **质量保证**：解的正确性有绝对保证

举例：

- **随机化选择算法**（找第k小元素）：
 - 随机选择枢轴
 - 总是返回正确的结果
 - 期望线性时间

蒙特卡罗算法：

特点：

1. **可能出错**：返回的解可能不正确
2. **运行时间确定**：通常有固定的时间界限
3. **错误概率可控**：可以通过参数调整错误概率
4. **效率稳定**：运行时间可预测

2019年

问题1

动态规划算法的两个基本要素是什么？

答案

最优子结构性质和子问题重叠性。

问题2

请回答：

- (1) 优先队列式分支限界法中活结点表采用什么数据结构实现？
- (2) 简单描述优先队列插入算法基本思想。

答案

- (1) 优先队列式分支限界法中活结点表通常采用 **堆** 数据结构实现，具体为：
 - 最小堆（当优先级为下界或成本函数值越小优先级越高）

- 最大堆（当优先级为上界或收益函数值越大优先级越高）

(2) 优先队列插入算法的基本思想（以最小堆为例）：

1. 将新元素插入到堆的末尾（完全二叉树的最后一个位置），以维持完全二叉树的结构。
2. 比较新元素与其父节点的优先级：
 - 若新元素优先级更高（值更小），则交换它们的位置。
3. 重复步骤2，直至新元素到达堆顶或满足堆序性质（即父节点优先级不高于新元素）。

问题3

解下列递推方程：

$$\begin{cases} x(n) = 3x(n-1) & , \text{当 } n > 1 \text{ 时} \\ x(1) = 4 & , \text{当 } n = 1 \text{ 时} \end{cases}$$

答案

当 $n > 1$ 时：

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 9x(n-2) \\ &= \dots \\ &= 3^{n-1}x(1) \\ &= 4 \cdot 3^{n-1} \end{aligned}$$

当 $n = 1$ 时，也满足 $x(n) = 4 \cdot 3^{n-1}$

综上，递推方程的解为：

$$x(n) = 4 \cdot 3^{n-1}$$

问题4

请填写下列程序中空缺部分（背包问题的贪心算法）

```
void Knapsack(int n,float M,float v[],float w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++)
    {
        float c=M;
        for (i=1;i<=n;i++)
        {
            if (w[i]>c) break;
            x[i]=1;
            ___(1)___;
        }
        ___(2)___;
    }
}
```

```
}  
}
```

答案

```
(1) c -= w[i]  
(2) x[i] = c / w[i]
```

2021年A

问题1

请简述用分治法如何实现合并排序。【5分】

答案

1. 把待排序数组均分为两半。
2. 对左右两个子数组分别递归地进行合并排序，直到子数组长度为 1。
3. 把已排序的左右子数组用一次线性时间的“归并”过程合并成一个有序数组（用两个指针从头比较并依次取小者）。

问题2

请简述贪心算法求解背包问题算法的基本思想，并解释为何0-1背包问题不能用贪心算法求解。【5分】

答案

贪心算法基本思想（用于分数背包）：对每个物品计算价值密度（单位重量价值） v_i/w_i ，按密度从高到低把物品放入背包；当遇到一个物品不能完全放下时允许**切分**，取能装下的部分并终止。该策略在允许切分的情形下是最优的。

为何 0-1 背包不能用同样的贪心策略：0-1 背包要求每件物品要么全部放入要么不放，不能切分。贪心按单位价值取最大的局部选择可能导致错过组合上更优的整体解。

问题3

请简述回溯法与分支限界法的异同。【5分】

答案

相同点

1. 二者都把问题建模为**解空间树**，通过枚举分支的方式搜索解空间。
2. 都常用**剪枝**以减少搜索量。
3. 都能用于组合问题的精确求解（可用于决策问题、优化问题）。

不同点

1. 目标侧重点：

- 回溯法多用于**可行解的枚举或找一个可行解**（判定/枚举问题）；
- 分支限界法主要用于**优化问题**，寻找最优解并利用界函数剪枝。

2. 剪枝依据不同：

- 回溯主要用**约束/可行性条件**（是否违反约束）剪枝；
- 分支限界除了可行性外还用**上下界（bound）**比较当前最优上界来剪枝。

3. 搜索策略常不同：

- 回溯通常采用**深度优先**实现并回溯；
- 分支限界既可用**队列**也常用**优先队列**按界值选择下一个扩展节点。

问题4

请简述用随机投点法计算圆周率。【5分】

答案

用蒙特卡洛（随机投点）估算 π 的基本方法：

1. 在单位正方形 $[0, 1] \times [0, 1]$ 内均匀地独立采样 N 个点 (x_i, y_i) 。
2. 计算落在单位四分之一圆（以原点为圆心，半径 1 的第一象限圆弧）内的点数：

$$M = \{i : x_i^2 + y_i^2 \leq 1\}.$$

3. 由于四分之一圆面积为 $\pi/4$ ，故用比例估计 π ：

$$\hat{\pi} = 4 \cdot \frac{M}{N}.$$

2021年B

问题1

请简述用分治法如何实现快速排序。【5分】

答案

1. 从待排序数组中选择一个枢轴元素（pivot），把数组重排为三部分：小于枢轴、等于枢轴、大于枢轴。
2. 对枢轴左侧子数组和右侧子数组分别递归进行快速排序。
3. 左右子数组排序完成后与枢轴拼接得到有序数组。

问题2

请简述Dijkstra单源最短路径算法的基本思想。【5分】

答案

1. 初始化：将源点 s 的距离 $dist[s] = 0$ ，其它顶点 $dist = \infty$ 。
2. 选择最小标签：反复从未固定的顶点中选择当前 $dist$ 最小的顶点 u ，并把它标为已确定最短路径的顶点。
3. 松弛：对 u 的每条出边 (u, v) ，尝试用经过 u 的路径更新 $dist[v]$ ：若 $dist[v] > dist[u] + w(u, v)$ 则设 $dist[v] = dist[u] + w(u, v)$ 并更新优先队列。
4. 终止：当所有顶点被固定或目标已固定时结束。

问题3

请简述回溯法的深度优先搜索策略。【5分】

答案

回溯法把解空间视作一棵决策树，采用深度优先搜索策略尽可能沿一条分支深入：

1. 从根节点开始，按某种顺序对当前决策变量尝试每一个候选值。
2. 每做一次选择就递归进入下一层，同时维护当前状态并检查局部约束。
3. 若满足剪枝函数，则撤销本次选择（回溯）并尝试下一个候选；若到达叶节点且满足目标，则记录解或停止。

问题4

设 $f(x)$ 是 $[0, 1]$ 上的连续函数，且 $0 \leq f(x) \leq 1$ ，请简述用随机投点法计算定积分 $I = \int_0^1 f(x) dx$ 。【5分】

答案

在 $x \in [0, 1], y \in [0, 1]$ 单位正方形内均匀地作投点试验，则随机点落在曲线下方的概率为

$$P\{y \leq f(x)\} = \int_0^1 \int_0^{f(x)} dy dx = \int_0^1 f(x) dx$$

假设向单位正方形内随机地投入 n 个点 (x_i, y_i) 。如果有 m 个点落入 G 内，则随机点落入 G 内的概率

$$I \approx \frac{m}{n}$$

此即为对定积分的估计。

2023年

问题1

请简述：贪心算法和动态规划算法的区别与联系。

答案

联系

1. **最优化目标**：两者都用于求解最优化问题，寻求问题的最优解。
2. **最优子结构**：两类算法都要求问题具有最优子结构性质，即问题的最优解包含其子问题的最优解。
3. **递推特性**：都是通过一系列递推决策来构建最终解，将复杂问题分解为更小的子问题。

区别

1. **决策策略**：
 - 贪心算法：每一步都做出当前看来最优的选择，且选择后不可更改（不可回溯）。
 - 动态规划：考虑所有可能的子问题解，通过组合子问题的最优解来得到原问题的最优解。
2. **状态记录**：
 - 贪心算法：通常不需要保存子问题的解，只关注当前最优选择。
 - 动态规划：需要存储已解决的子问题的解，避免重复计算（记忆化）。
3. **适用范围**：
 - 贪心算法：仅适用于具有贪心选择性质的问题，且通常只能得到近似解或特定条件下的最优解。
 - 动态规划：适用于具有重叠子问题和最优子结构的问题，能保证得到全局最优解。

问题2

请简述：队列式分支限界法与优先级队列式分支限界法的异同。

答案

相同点

1. **算法框架**：两者都属于分支限界法的具体实现，都通过系统搜索解空间树来求解最优化问题。
2. **剪枝策略**：都使用限界函数剪去不满足约束条件或不可能产生最优解的分支。
3. **数据结构**：都需要使用队列（或优先队列）来存储待处理的活结点，都采用先进先出的扩展方式（从队列中取出结点进行扩展）。

不同点

1. **结点选择策略**：
 - 队列式（FIFO）分支限界法：按照结点进入队列的顺序选择扩展结点，即先进先出原则。
 - 优先级队列式分支限界法：按照结点的优先级（如成本函数值、目标函数值等）选择扩展结点，每次选择优先级最高/最低的结点。
2. **搜索方式**：
 - 队列式分支限界法：类似于广度优先搜索，逐层遍历解空间树。
 - 优先级队列式分支限界法：类似于最佳优先搜索，朝着最有希望的方向深入搜索。

3. 求解效率：

- 队列式分支限界法：需要遍历更多结点才能找到最优解，但实现简单。
- 优先级队列式分支限界法：通常能更快找到最优解，因为优先扩展最有希望的结点，但需要维护优先队列，实现较复杂。

问题3

请简述：可使用哪类随机化算法求解线性时间元素选择问题，并介绍其基本求解思路。

答案

算法类别：舍伍德随机化算法

基本思路（步骤）：

1. **随机选枢轴**：从待选数组中随机挑一个元素作为枢轴 p 。
2. **一次划分（partition）**：按与 p 的大小关系把数组划分为两部分：小于 p 、大于等于 p 。记小于部分大小为 L 。
3. **递归选择**：若要找第 k 小：
 - 若 $k \leq L$ ，在“小于 p ”子数组中递归寻找第 k 小；
 - 否则在“大于 p ”子数组中递归寻找第 $k - L$ 小。
4. **终止**：当子数组很小直接返回。

问题4

请填写下列程序中空缺部分（递归回溯的算法框架）。

```
void backtrack (int t) {
    if(t>n) output(x);
    else {
        for(int i=f(n,t);i<=g(n,t);i++) {
            x[t]=h(i);
            if(constraint(t) && ___(1)___)
                ___(2)___;
        }
    }
}
```

答案

- (1) Bound(t)
- (2) backtrack(t+1)

问题5

请计算时间复杂度、已知某递归算法运行时间表达式如下，请展开求其时间复杂度（大O表示法）

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T\left(\frac{n}{2}\right) + O(n) & n > 1 \end{cases}$$

答案

当 $n > 1$ 时：

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{2}\right) + cn \\ &= 4\left(4T\left(\frac{n}{2^2}\right) + c\frac{n}{2}\right) + cn \\ &= 4^2T\left(\frac{n}{2^2}\right) + cn(1 + 2) \\ &= 4^2\left(4T\left(\frac{n}{2^3}\right) + c\frac{n}{2^2}\right) + cn(1 + 2) \\ &= \dots \\ &= 4^{\log_2 n}T(1) + cn \sum_{j=0}^{i-1} 2^j \\ &= n^2T(1) + cn(n - 1) \\ &= O(n^2) \end{aligned}$$

综上：

$$T(n) = O(n^2)$$

2024年A

问题1

请简述回溯法和分支限界法的相同点和不同点（本题 6 分）。

答案

见2021年A问题3

问题2

请简述贪心算法的两个重要性质（本题 6 分）。

答案

1. 贪心选择性质：所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。
2. 最优子结构性质：一个问题的最优解包含其子问题的最优解。

问题3

请简述拉斯维加斯（Las Vegas）算法和蒙特卡洛（Monte Carlo）算法的主要区别（本题 10 分）。

答案

1. 正确性保证：

- 拉斯维加斯算法：如果算法返回一个解，那么该解一定是正确的。
- 蒙特卡洛算法：算法返回的解可能不正确，但错误概率可以控制。

2. 运行时间特性：

- 拉斯维加斯算法：运行时间是随机的，期望时间复杂度可分析，但最坏情况下可能无法在有限时间内结束。
- 蒙特卡洛算法：运行时间通常是固定的，可以在预定时间内完成计算。

3. 输出结果：

- 拉斯维加斯算法：要么返回正确解，要么返回“无解”或运行超时。
- 蒙特卡洛算法：总是返回一个解，但该解有一定的错误概率。

4. 提高可靠性的方式不同

- 拉斯维加斯算法：通过允许算法继续运行或多次重试来获得结果，本质上消除错误。
- 蒙特卡洛算法：通过多次独立运行并取多数结果，来降低出错概率，但不能完全消除错误。

问题4

请填写下列程序中空缺部分（递归回溯搜索子集树的一般算法框架）（本题 8 分）。

```
void backtrack (int t)
{
    if (t>n) Output(x);
    else
        for (int i=0;i<l;i++)
        {
            ___(1)___;
            if (Constraint(t) && bound(t))
                ___(2)___;
        }
}
```

答案

```
(1) x[t] = i
(2) backtrack(t + 1)
```

2024年B

问题1

请简述什么是 NP 问题，请指出哪些问题是 NP 完全问题（要求举出四个以上）（本题 6 分）。

答案

什么是 NP 问题（简述）

一个判定问题属于 NP，当且仅当：对于任给的“是”实例，存在一个长度多项式的证据，使得可以在多项式时间内由确定性图灵机验证该证据从而确认该实例为“是”。等价地，NP 是那些能被非确定性多项式时间机器接受的判定问题集合。

NP 完全问题的定义（要点）

一个判定问题 L 是 NP-完全（NP-complete），当且仅当：

1. $L \in \text{NP}$ ，且
2. 对于任意 $L' \in \text{NP}$ ，存在多项式时间归约 $L' \leq_p L$ （即 L 对 NP 中所有问题都是至少与之等难的）。

换言之，NP-完全问题既在 NP 中又是 NP-难的（NP-hard）。

常见的 NP-完全问题（举例，至少四个）

- 布尔可满足性问题（SAT），以及其规范形式 3-SAT。
- 最大团判定（Clique）：判断图中是否存在大小至少 k 的团。
- 顶点覆盖判定（Vertex Cover）：判断图中是否存在大小不超过 k 的顶点覆盖。
- 哈密顿回路判定（Hamiltonian Cycle）：判断图中是否存在经过每个顶点恰好一次的回路。
- 旅行商决策形式（TSP decision）：是否存在总长度不超过 K 的哈密顿回路（加权图）。

问题2

请简述队列式分支限界法和优先队列分支限界法的区别（本题 6 分）。

答案

见2023年A问题2

问题3

请简述随机算法中拉斯维加斯算法的过程（本题 10 分）。

答案

基本过程步骤（概括性描述）

1. **引入随机性**：算法在内部通过随机数生成器做随机选择，这些随机选择影响后续控制流程或分支，但不会破坏答案的正确性。
2. **执行带随机性的计算过程**：基于随机选择执行一系列确定性操作（例如划分、递归、验证等）；若中途达到可接受的解则返回。
3. **终止与正确性**：当算法停止时，它返回正确解（或明确说明未找到，随后可选择重试）；正确性由算法设计保证，不依赖随机性。

问题4

请填写下列程序中空缺部分（递归回溯搜索排列树的一般算法框架）（本题 8 分）。

```
void backtrack(int t) {  
    if (t > n) Output(x);  
    else {  
        for (int i = ___(1)___; i < n; i++) {  
            ___(2)___;  
            if (Constraint(t) && Bound(t)) Backtrack(t + 1);  
            swap(x[t], x[i]);  
        }  
    }  
}
```

答案

(1) 0

(2) swap(x[t], x[i])